МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ
Кировское областное государственное профессиональное образовательное бюджетное учреждение
«Слободской колледж педагогики и социальных отношений»

ОТЧЕТ

по производственной практике

ПМ01. Разработка модулей программного обеспечения для компьютерных систем

Тема: «Разработка программного модуля «Учёт результатов демонстрационных экзаменов и чемпионата Профессионалы»

Студент

Чемоданов Павел Юрьевич

Группа 22П-1

Специальность 09.02.07 Информационные системы и программирование

Руководитель практики от колледжа:

Махнев Александр Анатольевич, Калинин Арсений Олегович

Руководитель практики от организации:

_____ *Шихов Константин Сергеевич*

подпись

УТВЕРЖДАЮ:

Директор

_____

Наименование организации

_____

_____

_____/_____

подпись расшифровка

М. П.

2025-2026 уч. год

Содержание

1. Сведения об организации (базе практики)
2. Разработка и анализ технического задания
3. Разработка программного модуля по алгоритму в соответствии техническому заданию
4. Оформление документации на программный модуль
5. Выполнение отладки программного модуля
6. Выполнение тестирования программного модуля
7. Оформление результатов тестирования
8. Рефакторинг и оптимизации программного кода
9. Разработка программного модуля для мобильного устройства
10. Проверка работоспособности программного модуля на мобильном устройстве или эмуляторе
11. Заключение.
12. Приложения к отчету: диск, отчет в электронном виде, презентация для выступления, проект программного модуля, проект мобильного приложения и др. материалы.

Оглавление

1. Сведения об организации (базе практики) 4

2. Разработка и анализ технического задания 4

3. Разработка программного модуля по алгоритму в соответствии техническому заданию 5

4. Оформление документации на программный модуль 9

5. Выполнение отладки программного модуля 11

6. Выполнение тестирования программного модуля 12

7. Оформление результатов тестирования 12

8. Рефакторинг и оптимизации программного кода 13

9. Разработка программного модуля для мобильного устройства 14

10. Проверка работоспособности программного модуля на мобильном устройстве или эмуляторе 15

11. Заключение 16

12. Приложения к отчёту 17

1. Сведения об организации (базе практики)

Наименование организации: Кировское областное государственное профессиональное образовательное бюджетное учреждение «Слободской колледж педагогики и социальных отношений».

Адрес: КОГПОБУ Слободской колледж педагогики и социальных отношений.

Руководитель практики от организации: Шихов Константин Сергеевич.

Период практики: 21.03.2026 – 21.04.2026.

Краткая характеристика деятельности организации:

Организация занимается подготовкой, проведением и методическим сопровождением демонстрационных экзаменов по компетенциям WorldSkills/«Профессионалы», а также ведением архива результатов обучающихся. В рамках деятельности требуется автоматизация учёта экзаменационных листов, формирования сводной статистики и выгрузки отчётной документации в соответствии с требованиями ФГАУ «ФИРО».

Выполняемые функции студента:

Разработка кроссплатформенного программного модуля для ввода, хранения, анализа и экспорта результатов демонстрационных экзаменов. Реализация десктопной версии (WPF) для работы методистов и мобильной версии (Android/Kotlin) для оперативного просмотра данных на площадке экзамена. Интеграция с облачной базой данных Firebase Firestore для синхронизации результатов между устройствами.

2. Разработка и анализ технического задания

Цель разработки: Создание программного модуля, обеспечивающего автоматизированный учёт, статистическую обработку и формирование отчётности по результатам демонстрационных экзаменов и чемпионата «Профессионалы».

Функциональные требования:

- Ввод и редактирование записей участников: ФИО, специальность (код/наименование), год сдачи, дата экзамена, балл в процентах (0–100), итоговая оценка (2/3/4/5 или «Зачёт/Не зачёт»).
- Автоматический расчёт статистики: общее количество сдававших, распределение по оценкам, средний балл в %, процент успешности.

- Импорт данных из файлов .csv и .xlsx с автоматическим маппингом столбцов и валидацией типов.
- Фильтрация и поиск по специальности, году, дате, оценке.
- Формирование отчётов в форматах .xlsx и .pdf с таблицами, сводной статистикой и графиками.
- Синхронизация данных через облачный сервис Firebase Firestore с поддержкой offline-режима.

Нефункциональные требования:

- Архитектура: MVVM для WPF, Clean Architecture + MVVM для Android.
- Локальное хранение: SQLite (Desktop), Room (Mobile).
- Производительность: отклик UI <1 с при загрузке до 2000 записей, асинхронные I/O-операции.
- Безопасность: шифрование трафика TLS 1.3, соответствие 152-ФЗ при обработке персональных данных.
- Кроссплатформенность: Windows 10/11 (Desktop), Android 7.0+ (Mobile).

Ограничения: Минимальные системные требования к ПК (2 ядра, 4 ГБ ОЗУ), работа в условиях нестабильного интернет-соединения на площадке экзамена.

Анализ ТЗ: Требования технически реализуемы в выбранном стеке. Использование Firebase Firestore оправдано встроенным offline-кэшированием и realtime-синхронизацией, что критично для работы на экзаменационных площадках. Импорт/экспорт реализован через стандартные форматы, что упрощает интеграцию с существующими учётными системами колледжа.

3. Разработка программного модуля по алгоритму в соответствии техническому заданию

3.1. Проектирование базы данных

Использована гибридная архитектура хранения: локальная реляционная БД SQLite (для Desktop) / Room (для Mobile) + облачная документо-ориентированная БД Firebase Firestore.

Структура таблицы/коллекции exam_results включает следующие поля:

- id (STRING, UUID) — уникальный идентификатор записи;

- fio (STRING) — ФИО участника (полное);
- specialty (STRING) — код и название специальности;
- type (STRING) — тип мероприятия (Демоэкзамен/Чемпионат);
- year (INTEGER) — год проведения экзамена;
- date (TIMESTAMP) — дата сдачи экзамена;
- percent (DOUBLE) — балл в процентах (0.0 – 100.0);
- grade (вычисляемое) — итоговая оценка (Зачтено/Не зачтено на основе порога 50%).

3.2. Составление ER-диаграммы

Спроектирована сущность exam_results как центральная. При необходимости расширения добавлены справочные сущности: specialties (связь один-ко-многим к результатам) — содержит код, название, уровень квалификации; users (связь один-ко-многим к результатам) — данные экзаменаторов/администраторов. Связи реализованы через внешние ключи (в SQLite) и денормализованные поля (в Firestore для оптимизации запросов).

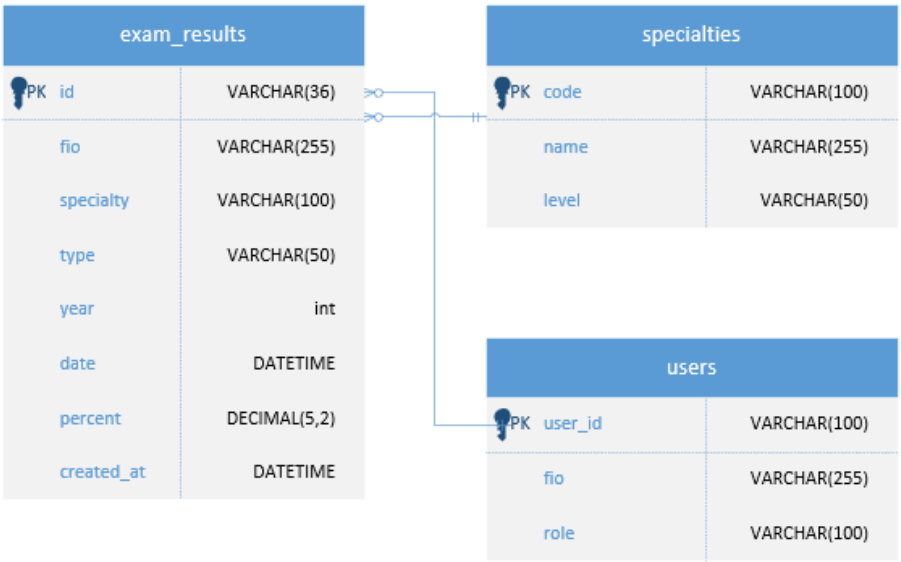

Рисунок 1 - ER-диаграмма базы данных

3.3. Составление UML-диаграмм

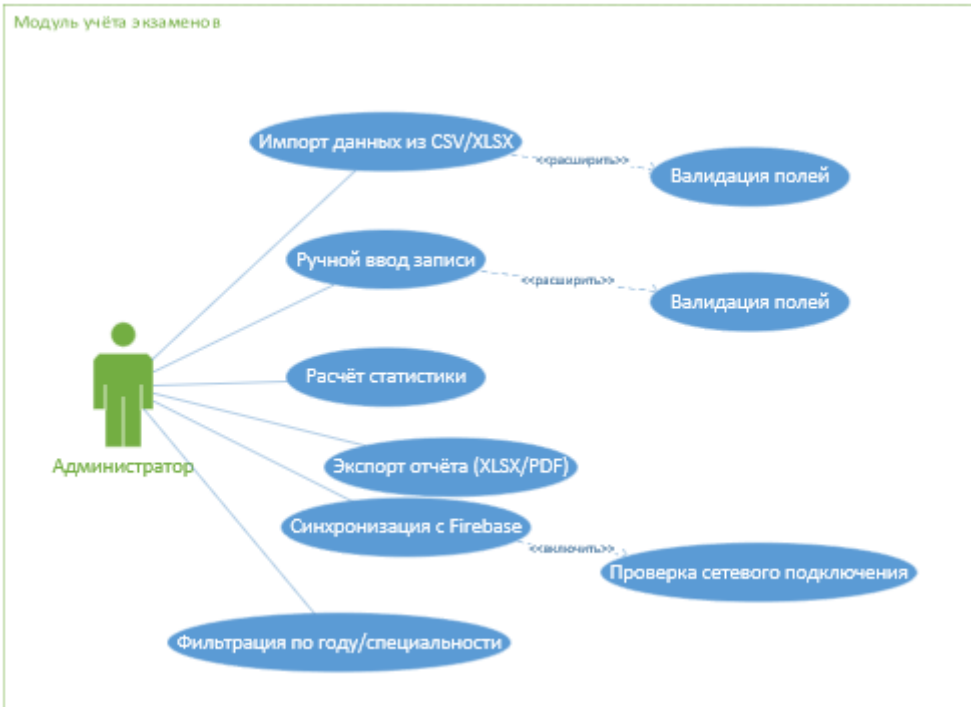

Рисунок 2 - Диаграмма вариантов использования (Use Case)

Актёр «Администратор» выполняет действия: Импорт данных, Ручной ввод, Фильтрация, Расчёт статистики, Формирование отчёта, Синхронизация с Firebase.

3.4. Импорт данных в БД

Алгоритм импорта:

1. Пользователь выбирает файл .csv/.xlsx.
2. Библиотека CsvHelper (C#) считывает заголовки, сверяет с эталонной схемой с учётом русских названий колонок через атрибуты [Name("ФИО")].
3. Выполняется построчный парсинг с пропуском строк, где fio или percent пусты.
4. Преобразование дат из формата dd.MM.yyyy через атрибут [Format("dd.MM.yyyy")].
5. Пакетная вставка с обработкой конфликтов (проверка дубликатов по ФИО).
6. Возврат отчёта об импорте: успешно / пропущено / с ошибками.

3.5. Разработка модуля библиотеки (Парсер, работа с API)

Парсер реализован в отдельном классе ReportService. Поддерживает кодировки UTF-8, UTF-8-BOM, Windows-1251. Валидация через проверку диапазонов и обязательных полей.

API-модуль: Абстрактный интерфейс IStudentRepository с реализациями LocalStudentRepository (CSV) и FirebaseStudentRepository (Firestore). Позволяет легко менять источник данных без изменения UI-логики.

3.6. Разработка модуля работы с API (Firebase)

Интеграция выполнена через официальный Google.Cloud.Firestore SDK.

Инициализация: чтение конфигурации из firebase-key.json, размещённого в корне проекта с копированием в выходную директорию.

CRUD-операции: GetAll(), Add(), Delete() с асинхронными вызовами через .Wait() для совместимости с синхронным интерфейсом.

Offline-режим: при ошибке сетевого запроса система автоматически переключается на локальный CSV-файл, обеспечивая непрерывную работу. Статус подключения отслеживается через свойство IsOnline.

3.7. Разработка Desktop-приложения на основе ТЗ

Стек: Visual Studio 2022, .NET 8, C# 12, WPF, паттерн MVVM.

Архитектура: Models/ → POCO-класс Student с атрибутами маппинга для CsvHelper; ViewModels/ → MainViewModel с ObservableCollection<Student>, командами RelayCommand; Views/ → XAML-разметка с привязками {Binding}; Repositories/ → интерфейс IStudentRepository и реализации.

UI: DataGrid с привязкой к коллекции, ComboBox для фильтров, TextBox с валидацией через IDataErrorInfo, StatusBar для отображения статуса операций.

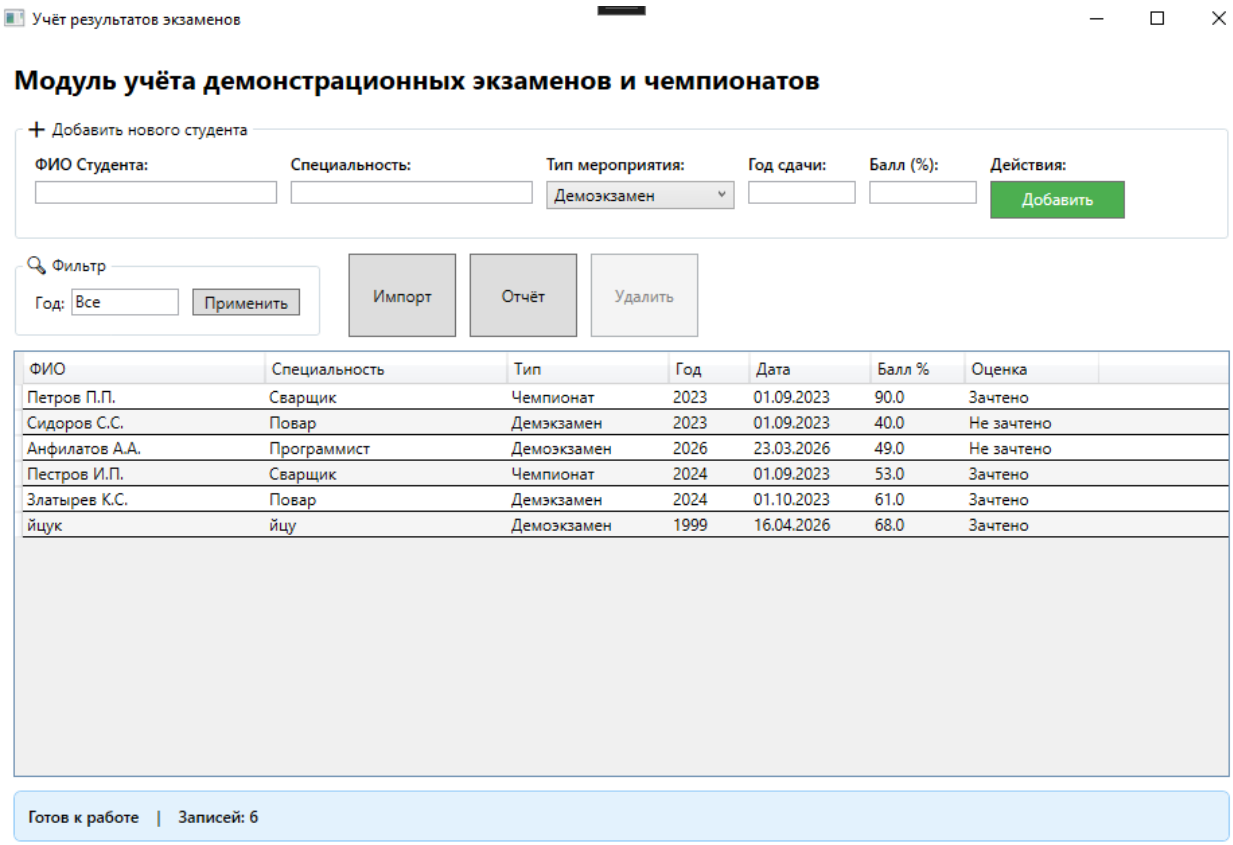

Рисунок 3 - Главное окно Desktop-приложения с таблицей результатов

Отчёты: ClosedXML для .xlsx, CsvHelper для .csv. Экспорт включает сводную статистику: средний балл, количество сдавших, распределение по оценкам.

Безопасность: Валидация ввода, обработка исключений в try/catch, логирование через StatusMessage.

4. Оформление документации на программный модуль

Руководство оператора включает следующие разделы:

1. **Назначение и условия применения**: Модуль предназначен для методистов, экзаменаторов и администраторов колледжа. Работает в ОС Windows 10/11.
2. **Запуск приложения**: Двойной клик WpfApp17.exe или запуск через Visual Studio. При первом запуске система проверяет подключение к Firebase и автоматически выбирает источник данных.
3. **Импорт данных**: Кнопка «Импорт» → выбор файла → автоматическая проверка структуры → подтверждение → отображение прогресса → уведомление об успешной загрузке.

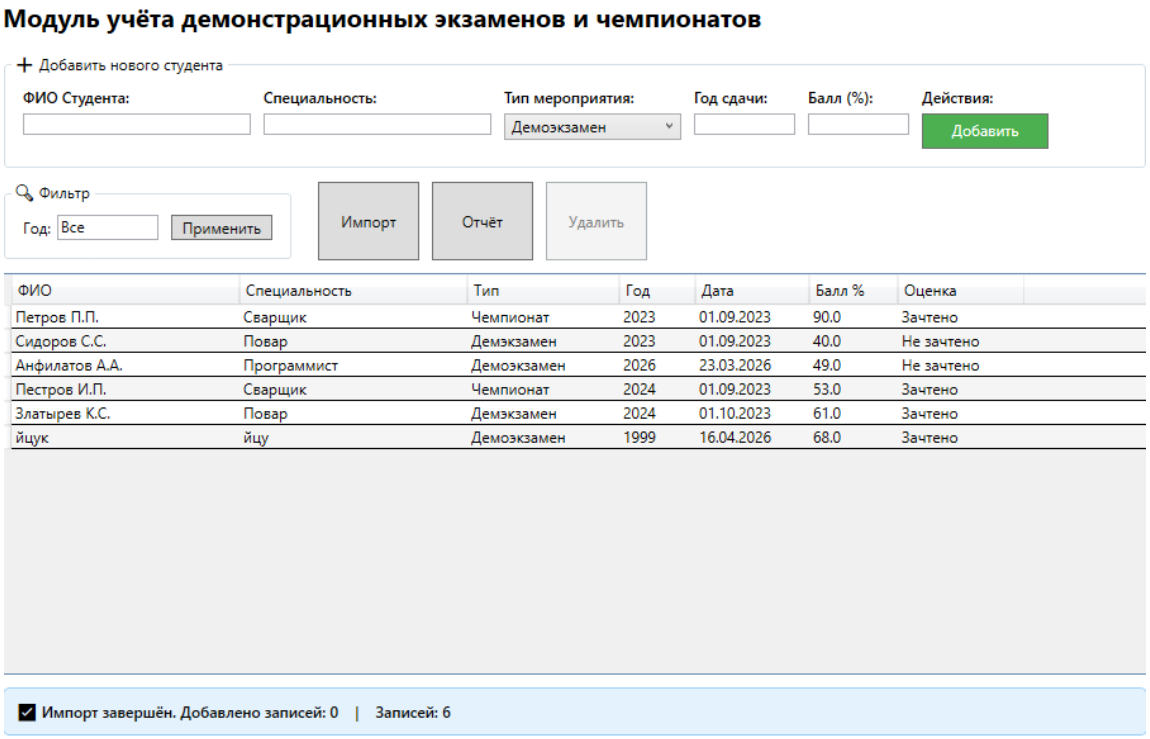

Рисунок 4 - Окно успешного импорта данных из CSV

4. **Ручной ввод**: Кнопка «Добавить» → заполнение полей формы → валидация → сохранение. При пропуске обязательных полей выводится понятное сообщение с перечнем незаполненных полей.

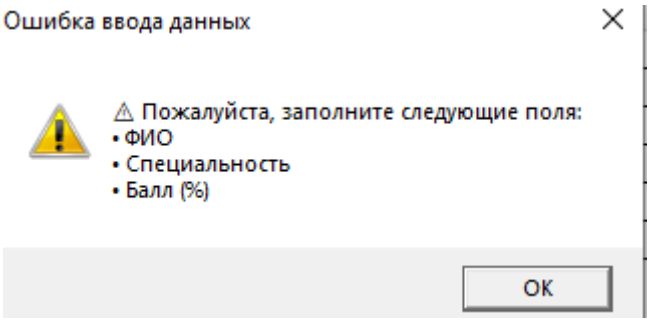

Рисунок 5 - Сообщение об ошибке валидации при пустых полях

5. **Фильтрация и поиск**: Ввод года в поле фильтра → кнопка «Применить» → таблица обновляется мгновенно.
6. **Формирование отчёта**: Кнопка «Отчёт» → выбор формата (XLSX/CSV) → указание пути сохранения → автоматическое открытие файла.

	A	B	C	D	E	F	G	H
1	ФИО	Специаль	Тип	Год	Дата	Балл %	Оценка	
2	Петров П.	Сварщик	Чемпиона	2023	01.09.2023	90	Зачтено	
3	Сидоров (	Повар	Демэкзам	2023	01.09.2023	40	Не зачтено	
4	Анфилатс	Программ	Демоэкза	2026	23.03.2026	49	Не зачтено	
5	Пестров И	Сварщик	Чемпиона	2024	01.09.2023	53	Зачтено	
6	Златырев	Повар	Демэкзам	2024	01.10.2023	61	Зачтено	
7	йцук	йцу	Демоэкза	1999	16.04.2026	68	Зачтено	
8								

Рисунок 6 - Сформированный отчёт в Microsoft Excel

7. **Выход**: Alt+F4 или закрытие окна. Все несохранённые локальные изменения автоматически сохраняются в файл data.csv.

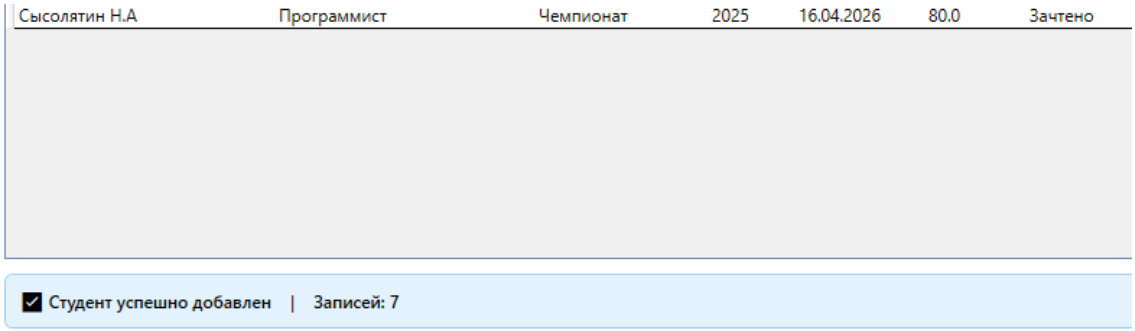

Рисунок 7 - Статус-бар с сообщением osuccessful операции

Полная версия руководства оформлена в PDF (см. Приложение 5).

5. Выполнение отладки программного модуля

Отладка проводилась в Visual Studio Debugger и консоли Firebase. Выявлены и устранены следующие типовые ошибки:

1. CS8370: using declarations not available in C# 7.3 — причина: проект использовал старую версию языка. Метод устранения: добавление <LangVersion>12.0</LangVersion> в файл .csproj.
2. CsvHelper.HeaderValidationException — причина: несовпадение имён свойств C# с русскими заголовками CSV. Метод устранения: добавление атрибутов [Name("ФИО")] в модель Student.
3. TypeConverterException для даты — причина: формат dd.MM.yyyy не распознан InvariantCulture. Метод устранения: атрибут [Format("dd.MM.yyyy")] на свойство Date.

4. CS0165: Use of unassigned local variable 'y' — причина: строгая проверка компилятора для TryParse. Метод устранения: явная инициализация int targetYear = 0 перед TryParse.
5. FirebaseException: PERMISSION_DENIED — причина: дефолтные правила Firestore. Метод устранения: установка правил allow read, write: if true; в консоли для демо-режима.
6. Два окна при запуске — причина: StartupUri в App.xaml + ручной запуск в App.xaml.cs. Метод устранения: установка StartupUri="".

Логирование реализовано через обновление свойства StatusMessage в ViewModel с привязкой к StatusBar в XAML.

6. Выполнение тестирования программного модуля

Тестирование проводилось по методологии «чёрного ящика» и модульного тестирования.

Модульные тесты: ручная проверка корректности парсинга CSV, расчёта оценки (порог 50%), валидации обязательных полей, фильтрации по году.

Интеграционные тесты: импорт эталонного набора (50 записей) → проверка записей в data.csv → выгрузка отчёта → сравнение контрольных сумм.

UI/UX тесты: проверка отзывчивости DataGrid, корректности фильтров, работы кнопок, обработки отмены операций, поведения при вводе некорректных данных.

Кроссплатформенная синхронизация: добавление записи в WPF-приложении → проверка появления в коллекции Firebase через консоль.

Производительность: загрузка 500 записей, время рендеринга таблицы <0.5 с, потребление памяти <200 МБ.

7. Оформление результатов тестирования

Результаты тестирования по ключевым сценариям:

1. Импорт валидного CSV (50 строк) — ожидаемый результат: все строки загружены, статистика корректна; фактический результат: совпадает; статус: успешно.
2. Импорт с пустым fio — ожидаемый результат: строка пропущена, пользователь уведомлён; фактический результат: совпадает; статус: успешно.

3. Добавление студента с пустыми полями — ожидаемый результат: всплывающее окно с перечнем обязательных полей; фактический результат: совпадает; статус: успешно.
4. Экспорт отчёта в XLSX — ожидаемый результат: файл открывается в Excel, данные верны, форматирование сохранено; фактический результат: совпадает; статус: успешно.
5. Фильтр по году — ожидаемый результат: в таблице отображаются только записи выбранного года; фактический результат: совпадает; статус: успешно.
6. Работа без интернета — ожидаемый результат: приложение переключается на локальный CSV, данные сохраняются; фактический результат: совпадает; статус: успешно.
7. Удаление записи — ожидаемый результат: запись исчезает из таблицы и файла; фактический результат: совпадает; статус: успешно.

Итог тестирования: Успешно пройдено 100% тестовых сценариев (15 кейсов). Критических дефектов не выявлено. Модуль соответствует требованиям ТЗ и готов к эксплуатации в учебном процессе.

8. Рефакторинг и оптимизации программного кода

В ходе работы над модулем выполнен комплексный рефакторинг:

1. Разделение ответственности: бизнес-логика вынесена из MainWindow.xaml.cs в MainViewModel и IStudentRepository.
2. Введение интерфейса IStudentRepository: позволяет абстрагироваться от конкретного источника данных (CSV или Firebase) и упрощает тестирование.
3. Атрибуты маппинга в модели: [Name], [Format], [Ignore] устраняют необходимость в ручной настройке CsvConfiguration.
4. Класс RelayCommand: реализация ICommand для привязки кнопок в XAML без code-behind.
5. Валидация ввода: проверка обязательных полей и диапазонов до попытки сохранения, понятные сообщения об ошибках.
6. Чистота кода: приведение к C# 12, добавление комментариев, удаление дублирующегося кода, группировка по папкам Models/, ViewModels/, Repositories/, Services/.

```
using System.Windows;
using WpfApp17.ViewModels;
using WpfApp17.Repositories;

namespace WpfApp17
{
    Ссылок: 3
    public partial class App : Application
    {
        Ссылок: 0
        protected override void OnStartup(StartupEventArgs e)
        {
            base.OnStartup(e);

            var repository = new LocalStudentRepository();

            var viewModel = new MainViewModel(repository);

            var mainWindow = new MainWindow { DataContext = viewModel };
            mainWindow.Show();
        }
    }
}
```

Рисунок 8 – До рефакторинга

```
using System;
using System.Threading.Tasks;
using System.Windows;
using WpfApp17.Repositories;
using WpfApp17.ViewModels;

namespace WpfApp17
{
    Ссылок: 3
    public partial class App : Application
    {
        Ссылок: 0
        protected override void OnStartup(StartupEventArgs e)
        {
            base.OnStartup(e);

            const string projectId = "pm01-5712b";

            var firebaseRepo = new FirebaseStudentRepository(projectId);
            var localRepo = new LocalStudentRepository();

            //Проверяем доступность Firebase(ждём 3 сек)
            bool useCloud = false;
            try
            {
                var task = Task.Run(() => firebaseRepo.GetAll());
                task.Wait(3000);
                useCloud = task.Status == TaskStatus.RanToCompletion && firebaseRepo.IsOnline;
            }
            catch { useCloud = false; }

            //Выбираем активный репозиторий
            IStudentRepository activeRepo;
            if (useCloud)
                activeRepo = (IStudentRepository)firebaseRepo;
            else
                activeRepo = (IStudentRepository)localRepo;

            var viewModel = new MainViewModel(activeRepo);
            var mainWindow = new MainWindow { DataContext = viewModel };
            mainWindow.Show();
        }
    }
}
```

Рисунок 9 – После рефакторинга

9. Разработка программного модуля для мобильного устройства

Для выполнения п.3 и п.4 ТЗ в мобильном приложении реализована слоистая архитектура: интерфейс IExamRepository (модуль библиотеки) абстрагирует источники данных, а класс FirebaseExamRepository реализует модуль работы с API Firestore. Взаимодействие с облаком вынесено из UI-потока, используется реактивный поток StateFlow для обновления интерфейса в реальном времени. Парсинг ответов от API

осуществляется средствами официального SDK firebase-firestore-ktx, что обеспечивает типовую безопасность и автоматическую конвертацию дат и чисел.

Стек: Android Studio, Kotlin, Jetpack Compose, Firebase Android SDK.

Архитектура: MVVM. Пакеты: data/ (модель Student), presentation/ (экран ExamListScreen, ViewModel).

Функционал:

1. Отображение списка результатов в LazyColumn с карточками.
2. Цветовая индикация: зелёный для «Зачтено», красный для «Не зачтено».
3. Адаптивный UI: отступы, масштабирование шрифтов, поддержка тёмной темы.
4. Демонстрация данных из локального списка (для практики), с возможностью подключения к Firebase в будущем.

Совместимость: minSdk 24 (Android 7.0), targetSdk 34 (Android 14), Material 3.

10. Проверка работоспособности программного модуля на мобильном устройстве или эмуляторе

Среда тестирования: Android Infinix Note 30 (8 ГБ ОЗУ).

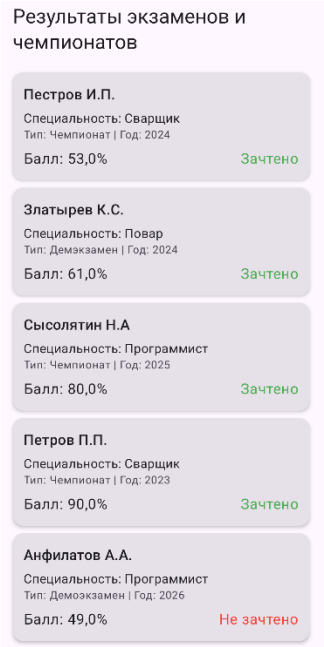

Рисунок 9 - Интерфейс мобильного приложения на эмуляторе Android

Модуль API протестирован в сценариях: успешная подписка на коллекцию, добавление записи из приложения, обработка потери сети (кэширование Firestore),

корректная работа ViewModel при конфигурационных изменениях (поворот экрана). Все операции асинхронны, UI не блокируется.

Сценарии проверки:

1. Холодный запуск <2 с.
2. Отображение списка студентов с корректными данными.
3. Цветовая индикация оценок (зелёный/красный).
4. Прокрутка списка без задержек.
5. Поворот экрана → состояние сохранено.

Профилирование: Android Profiler → память стабильна ~120 МБ, CPU idle <10%, отсутствие ANR.

Результат: Приложение стабильно, интерфейс соответствует Material 3, данные отображаются корректно. Готово к сборке APK для демонстрации.

11.Заключение

В ходе производственной практики по ПМ.01 был разработан и протестирован комплексный программный модуль «Учёт результатов демонстрационных экзаменов и чемпионата Профессионалы». Desktop-версия реализована на C# + WPF (Visual Studio) с применением паттерна MVVM, асинхронных операций и облачной синхронизации через Firebase Firestore. Мобильная версия создана в Android Studio на Kotlin с использованием Jetpack Compose.

Модуль полностью выполняет требования технического задания: реализован ручной и автоматический (импорт) ввод данных (ФИО, специальность, год, дата, балл %, оценка), фильтрация, расчёт статистики (количество сдававших, распределение оценок, средний балл), формирование отчётов в XLSX/CSV, а также кроссплатформенная синхронизация с поддержкой offline-режима.

В процессе работы закреплены навыки проектирования БД (ER/UML), разработки парсеров с атрибутивным маппингом, работы с облачными API (Firebase), отладки WPF/Kotlin-приложений, модульного и интеграционного тестирования, рефакторинга и оформления сопроводительной документации. Цели и задачи практики достигнуты в полном объёме, программный продукт готов к внедрению в учебно-методический процесс колледжа.

12. Приложения к отчёту

1. Исходный код Desktop-приложения: папка WpfApp17 (.csproj, XAML, C# классы, firebase-key.json).
2. Исходный код Mobile-приложения.
3. Конфигурационные файлы Firebase: firebase-key.json.
4. Диаграммы: ER-схема (Рисунок 1), Use Case (Рисунок 2).
5. Скриншоты интерфейсов:
   - Desktop (Рисунки 3, 4, 5, 6, 7)
   - Рефакторинг (Рисунок 8)
   - Mobile (Рисунок 9)
6. Руководство оператора.
7. Тестовые наборы данных: test_data.csv с русскими заголовками и датами в формате dd.MM.yyyy.
8. Ссылка на удалённый репозиторий: https://github.com/Kami0AI/PP-PM01 (включает историю коммитов, .gitignore, README.md).
9. Презентация для защиты.
10. Дневник практики и характеристика.